

In C, variables declared inside a function can be either automatic or static, and they differ mainly in lifetime and behavior. Automatic variables (declared with the `auto` keyword or by default) are created when the function is called and destroyed when the function exits, meaning they do not retain their values between function calls; each time the function runs, they are reinitialized. In contrast, static variables (declared with the `static` keyword) are initialized only once and persist for the entire duration of the program, even though their scope remains limited to the function in which they are defined. This persistence allows static variables to retain their previous values between function calls, making them useful for tasks like counting how many times a function has been executed or maintaining state information. As a result, automatic variables ensure fresh computation each time, while static variables enable functions to have memory of past executions, significantly affecting how the function behaves across multiple calls.